CSPLib prob018: Water Bucket Problem

The water bucket problem asks how to reach specified amounts using containers of fixed capacities. This project implements CSPLib problem 018 with a generalized number of buckets and a scheduling extension.

Implemented extension

Instances support two or more buckets, arbitrary capacities and starting amounts, and partial goals (-1 means any final amount). The sequential model finds a sequence of complete pours. The scheduling model adds complete FILL and EMPTY operations, permits parallel actions only on disjoint buckets, and minimizes makespan.

A pour transfers the maximum possible amount, stopping only when the source is empty or the destination is full. FILL fills a bucket to capacity; EMPTY drains it completely. No action transfers water in one-litre increments.

Implementation and example

proj.py uses water_bucket.mzn. proj_scheduling.py uses water_bucket_scheduling.mzn. visualize_schedule.py animates schedules produced by proj_scheduling.py.

For capacities [8, 5, 3], initial [8, 0, 0] and goal [4, 4, -1], the sequential solver returns this seven-pour solution:

```
[8, 0, 0] -> [3, 5, 0] -> [3, 2, 3] -> [6, 2, 0] ->
[6, 0, 2] -> [1, 5, 2] -> [1, 4, 3] -> [4, 4, 0]
```

Software and running the project

Development environment: Python 3.10.12, MiniZinc CLI 2.5.3 and Gecode 6.2.0. The project README targets MiniZinc >= 2.10.1; files/requirements.txt specifies the MiniZinc Python package >= 0.9.0. Install MiniZinc with Gecode and run the following from the repository root:

```
python3 -m pip install -r files/requirements.txt
python3 files/proj.py --file files/instances/classic_8_5_3.json
python3 files/proj_scheduling.py --file files/instances/classic_8_5_3.json
python3 files/visualize_schedule.py --file files/instances/classic_8_5_3.json
python3 -m unittest discover -s files/tests -v
```

Web resources and Generative AI reflection

The project made use of generative AI assistance to help translate the mathematical idea into Python and MiniZinc code, especially validation logic. It was also used to help with the visualization part (visualize_schedule.py). However, all generated code was reviewed, adjusted, and tested manually against the repository's own reference solver and benchmark instances.